

Universidad Nacional Autónoma de México

FES Cuautitlán — Ingeniería en Telecomunicaciones, Sistemas y Electrónica

Diseño de un sistema basado en el microprocesador Z80
Microprocesadores — Actividad I

Alumno: Brayan Ivan Noriega Guzman

19 de septiembre de 2026

Resumen

Este documento describe el diseño de un sistema mínimo basado en el microprocesador Z80. El diseño de hardware contempla 16 Kbytes de memoria ROM (EEPROM 28C256) iniciando en la dirección 0000h, 16 Kbytes de memoria RAM (dos SRAM 6264) iniciando en la dirección F800h, y 6 puertos de entrada/salida implementados mediante dos interfaces PPI 8255 mapeadas a partir de la dirección 40h. En cuanto al software, se desarrolló un programa en lenguaje ensamblador que solicita al usuario su nombre y apellidos, valida estrictamente el ingreso de letras y espacios (rechazando números y símbolos), y cuenta el número total de letras ingresadas. El diseño de hardware se documenta mediante KiCad, y el software fue validado en el simulador Z80_workbench.

Introducción

El microprocesador Z80, presentado por Zilog, es un procesador de 8 bits ampliamente reconocido por su arquitectura CISC. Por su accesibilidad a los buses de direcciones, datos y señales de control, sigue siendo una herramienta didáctica fundamental para comprender la arquitectura de computadoras.

El objetivo de este proyecto es diseñar una computadora funcional a nivel de hardware y software. Esto requiere decodificar un mapa de memoria con rangos operativos estrictos utilizando decodificadores (como el 74LS138) y compuertas lógicas, asegurando la correcta habilitación de las memorias y periféricos. Asimismo, se busca la integración de este hardware con un programa en ensamblador capaz de controlar el flujo de entrada y salida, interactuando con una pantalla y un teclado.

Funcionamiento del sistema

Microprocesador

Se utilizó el microprocesador Z80 (Z84C00). Posee un bus de datos bidireccional de 8 líneas (D0-D7) y un bus de direcciones de 16 líneas (A0-A15), lo que le permite direccionar hasta 64 Kbytes de memoria y 256 puertos de E/S. Las señales de control principales utilizadas son:

- **/MREQ (Memory Request):** Activa en baja, indica una dirección válida para memoria.
- **/IORQ (I/O Request):** Activa en baja, indica una dirección válida para un periférico.
- **/RD y /WR:** Indican si el ciclo es de lectura o escritura.

Memorias

- **ROM (28C256):** Memoria EEPROM utilizada para almacenar el código ejecutable. Tiene una capacidad de 32 Kbytes, pero el diseño direcciona específicamente 16 Kbytes conectando las líneas correspondientes. Su pin de escritura se inhabilita para que el procesador solo realice ciclos de lectura de instrucciones.
- **SRAM (6264):** Memorias estáticas volátiles. Para cumplir con el requerimiento de 16 Kbytes de memoria de datos, se utilizan dos circuitos integrados 6264 (de 8 Kbytes cada uno). Permiten ciclos de lectura y escritura para almacenar las variables del programa y la pila.

Interfaz para periféricos

PPI (8255): La interfaz periférica programable es un controlador de entrada/salida. Cada 8255 proporciona 24 líneas organizadas en tres puertos de 8 bits (A, B y C). Se incorporaron dos integrados 8255 para cumplir con el requisito de 6 puertos en total. Para su direccionamiento utilizan los pines A0 y A1, y se activan mediante la señal /IORQ del Z80.

Diseño del sistema

Mapa de Memoria y Decodificación

El diseño del hardware cumple estrictamente con las direcciones solicitadas:

1. **ROM (0000h a 3FFFh):** Se emplea lógica de decodificación evaluando los pines A14 y A15 del bus de direcciones (ambos en nivel bajo) en conjunto con la señal /MREQ para habilitar la EEPROM.
2. **RAM (F800h en adelante):** La dirección base F800h (1111 1000 0000 0000 en binario) se decodifica evaluando los pines A11, A12, A13, A14 y A15 (todos en nivel alto). Esta señal, validada por /MREQ, habilita el bloque de 16 Kbytes de memoria SRAM.
3. **Puertos (40h):** Se utiliza un decodificador 74LS138 habilitado por /IORQ. Evaluando los pines A5, A6 y A7, se activa el primer PPI 8255 en la dirección 40h, y mediante otra salida del decodificador, el segundo PPI para expansión.

Diseño de Software (Algoritmo)

La lógica del programa estructurado en lenguaje ensamblador sigue el siguiente flujo:

1. Inicializar los puertos del PPI1 (Puerto A como salida para pantalla, Puerto C como entrada para teclado).
2. Mostrar en pantalla el mensaje de solicitud de nombre y apellidos.
3. Inicializar el registro contador de letras en 0.
4. **Ciclo Principal de Lectura:**
 - Leer el puerto C (teclado).
 - **Si** es la tecla [ENTER], saltar a la rutina de finalización.
 - **Si** es la tecla [ESPACIO], imprimir en pantalla e ignorar el conteo.
 - **Si** el carácter está en el rango ASCII válido de letras (mayúsculas o minúsculas), imprimir en pantalla e incrementar el registro contador.
 - **De lo contrario**, imprimir mensaje de ".Error: carácter inválido".
5. **Rutina de finalización:** Imprimir la cadena "Total de letras:" seguida de la conversión hexadecimal/decimal del registro contador.
6. Fin del programa (HALT).

Análisis de los resultados

El diseño de la lógica combinacional garantizó el correcto mapeo de las memorias y los periféricos en los rangos establecidos por las instrucciones. A nivel de software, la simulación en Z80_workbench validó con éxito el algoritmo planteado. Utilizando operaciones de comparación (CP) y saltos condicionales (JR C, JR NC, JR Z), el programa logró filtrar la tabla ASCII para admitir únicamente letras (41h-5Ah, 61h-7Ah) y espacios (20h), rechazando números y caracteres especiales con su respectivo mensaje de error.

Conclusión

El desarrollo de esta actividad permitió aplicar los fundamentos de la arquitectura de computadoras diseñando un sistema basado en el Z80 desde cero. Se demostró la capacidad de decodificar mapas de memoria específicos mediante compuertas lógicas y decodificadores, habilitando la ROM, la RAM y los puertos PPI de manera controlada. Asimismo, la elaboración del programa en ensamblador reforzó el manejo de instrucciones a bajo nivel, manipulación de registros y control del hardware, cumpliendo satisfactoriamente todos los requisitos de validación de entrada solicitados.

Referencias

- 1 Zilog, Inc., *Z80 CPU User Manual (UM008011)*. San Jose, CA, EE. UU.: Zilog.
- 2 Atmel Corporation, *AT28C256: 256K (32K x 8) Paged Parallel EEPROM Datasheet*. San Jose, CA, EE. UU.: Atmel.
- 3 Hitachi (Renesas), *HM6264 Series: 8192-word $\times$ 8-bit CMOS Static RAM Datasheet*. Tokio, Japón: Hitachi.
- 4 Intel Corporation, *8255A Programmable Peripheral Interface Datasheet*. Santa Clara, CA, EE. UU.: Intel.